MERN VPS Deployment

Dockerized Node API + Nginx reverse proxy + React static hosting + DuckDNS + Let's Encrypt SSL

Version: Feb 2026 (generated February 01, 2026)

What this document assumes

• Your repo has two folders: **server/** (Node/Express API on port 5000) and **client/** (React/Vite app that builds to **client/dist**).

• Database is **MongoDB Atlas** (so no Mongo container in production).

• You are deploying on an **Ubuntu** VPS as **root**.

• Domain is a DuckDNS subdomain (example used below: **mernvpsdeploy.duckdns.org**).

Variables to replace

• **YOUR_VPS_IP** - the public IPv4 of your VPS

• **YOUR_DUCKDNS_TOKEN** - your DuckDNS token

• **YOUR_GITHUB_TOKEN** - a GitHub Personal Access Token (PAT) with repo read access (or use SSH keys instead)

• **DOMAIN** - your DuckDNS domain, e.g. mernvpsdeploy.duckdns.org

Deployment steps

Follow the same order as your checklist. Each step includes what it does and what you should expect.

Step 1 - Create backend Dockerfile and .dockerignore

```
# server/Dockerfile
FROM node:20-alpine

WORKDIR /app

COPY package*.json ./
RUN npm ci --omit=dev

COPY . .

ENV NODE_ENV=production
EXPOSE 5000

CMD ["node", "src/index.js"]

# server/.dockerignore
node_modules
npm-debug.log
.DS_Store
.env
dist
build
coverage
.git
```

This packages your Node API into a repeatable, production image. **npm ci --omit=dev** installs only production dependencies for smaller, safer images.

The **.dockerignore** prevents copying secrets and heavy folders (like node_modules) into the image build context.

Note: If your entry file is not **src/index.js**, update the CMD accordingly.

Note: Keep **server/.env** out of Git; it will be injected via docker-compose using **env_file**.

Step 2 - Create docker-compose.prod.yml and .gitignore

```
# docker-compose.prod.yml
services:
  api:
    build:
      context: ./server
      dockerfile: Dockerfile
    container_name: mern-api
    restart: unless-stopped
    env_file:
      - ./server/.env
    ports:
      - "5000:5000"  # Step 14 will change this to 127.0.0.1:5000:5000

.gitignore (root)
# Node
node_modules
npm-debug.log
pnpm-debug.log
yarn-error.log

# Env / secrets
.env
*.env
server/.env

# Builds
client/dist
server/dist
build

# OS / editor
.DS_Store
.vscode
```

docker-compose.prod.yml defines how the API container is built and run in production.

The initial port mapping exposes the API on port 5000 for testing. Later you will bind it to 127.0.0.1 so it is not publicly reachable, and only Nginx can reach it.

.gitignore keeps generated files and secrets out of Git history.

Step 3 - SSH into your VPS

```
ssh root@YOUR_VPS_IP
```

This opens a remote shell on the VPS. All remaining commands (Step 4 onwards) run on the VPS unless stated otherwise.

Step 4 - Update and upgrade OS packages

```
apt update -y && apt upgrade -y
```

Updates the package index and installs security/bugfix updates. Doing this first avoids installing outdated packages later.

Note: If the VPS prompts about restarting services, accept the defaults unless you know you need custom behavior.

Step 5 - Install core tools (Git, Curl, UFW, Nginx)

```
apt install -y git curl ufw nginx
```

Installs the tools you will use in this deployment: Git (clone repo), Curl (HTTP tests), UFW (firewall), and Nginx (reverse proxy + static hosting).

Step 6 - Configure firewall (UFW) for SSH and web traffic

```
ufw allow OpenSSH
ufw allow 'Nginx Full'
ufw enable
ufw status
```

Allows SSH (port 22) so you do not lock yourself out, and allows Nginx web traffic (ports 80 and 443).

ufw status confirms rules are active.

Note: If you use a custom SSH port, allow that port before enabling UFW.

Step 7 - Install Docker Engine and Docker Compose

```
curl -fsSL https://get.docker.com | sh
docker --version
docker compose version
```

Installs Docker using Docker's official convenience script.

The version checks confirm Docker is installed and the Compose plugin is available.

Note: You are running as root, so you do not need to add a user to the docker group.

Step 8 - Enable Docker to start on boot

```
systemctl enable --now docker
```

Starts Docker immediately and enables it to auto-start after VPS reboots.

Step 9 - Clone your repo into /opt

```
mkdir -p /opt
cd /opt
git clone
https://sangammukherjee:YOUR_GITHUB_TOKEN@github.com/sangammukherjee/mern-deployment.git
mern-vps-deployment
cd mern-vps-deployment
```

Keeps deployments under **/opt** (common location for app code on servers).

Clones your GitHub repository into a folder named **mern-vps-deployment**.

Note: Putting a token in the clone URL works, but it can leak via shell history. Prefer SSH keys for long-term use.

Step 10 - Create/edit server environment variables

```
nano server/.env
```

Adds production environment variables for the API container (Mongo Atlas URI, secrets, etc.). Docker Compose will load this file using **env_file**.

Note: Never commit this file.

Note: If your API reads PORT from env, keep it as 5000 to match the container port mapping.

Step 11 - Confirm docker-compose.prod.yml settings

```
nano docker-compose.prod.yml
```

This is where you confirm the **build context**, **env_file** path, and **ports** mapping are correct for your repo structure.

Step 12 - Build and start the API container

```
docker compose -f docker-compose.prod.yml up -d --build
docker compose -f docker-compose.prod.yml ps
```

Builds the API image (from server/Dockerfile) and starts the container in detached mode.

ps should show the container as **Up** and mapping port 5000.

Step 13 - Test the API locally on the VPS

```
curl http://localhost:5000/api/notes
```

This is a direct test to the container port mapping. If it fails here, fix Docker/API before touching Nginx or DNS.

Note: If your route is different, replace **/api/notes** with your real endpoint.

Step 14 - Restrict API port to localhost only (recommended)

```
nano docker-compose.prod.yml

# Change ports to:
#   - "127.0.0.1:5000:5000"
```

Binding to **127.0.0.1** prevents the API from being accessed directly from the internet. Nginx (running on the same VPS) will proxy requests to it.

Step 15 - Restart the container to apply the port change

```
docker compose -f docker-compose.prod.yml down
docker compose -f docker-compose.prod.yml up -d --build
```

Recreates containers using the updated compose file. After this, the API should no longer be reachable on the public IP:5000.

Step 16 - Re-test the API via loopback

```
curl http://127.0.0.1:5000/api/notes
```

Confirms the API is still working, but only locally on the VPS.

Step 17 - Point DuckDNS domain to your VPS IP

```
curl "https://www.duckdns.org/update?domains=mernvpsdeploy&token=YOUR_DUCKDNS_TOKEN&ip=Y
VPS_IP"
```

Updates DuckDNS so **mernvpsdeploy.duckdns.org** resolves to your VPS IP.

The response is typically **OK** on success.

Note: Use your own domain name and token. The IP must match your VPS public IPv4.

Step 18 - Verify DNS points to the VPS

```
apt install -y dnsutils
dig +short mernvpsdeploy.duckdns.org
```

Installs **dig** and checks DNS resolution. You should see your VPS IP returned.

Note: If it returns nothing, wait a bit and re-run; DNS propagation can take some time.

Step 19 - Start Nginx and confirm it is installed

```
systemctl enable --now nginx
nginx -v
```

Starts Nginx now and enables it to start automatically after reboot.

nginx -v prints the installed Nginx version.

Step 20 - Create Nginx site config (API proxy first)

```
nano /etc/nginx/sites-available/mern-vps-deployment

# /etc/nginx/sites-available/mern-vps-deployment
server {
  listen 80;
  server_name mernvpsdeploy.duckdns.org;

  # API reverse proxy
  location /api/ {
    proxy_pass http://127.0.0.1:5000;
    proxy_http_version 1.1;

    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
  }

  # For now, return a simple response on /
  location / {
    return 200 "Nginx is running. Frontend will be added later.\n";
    add_header Content-Type text/plain;
  }
}
```

Creates a dedicated Nginx site file. At this stage, Nginx proxies **/api** to the Dockerized API (127.0.0.1:5000).

Keeping the API behind Nginx lets you control TLS, logging, and rate limits in one place.

Step 21 - Enable the Nginx site and reload

```
rm -f /etc/nginx/sites-enabled/default
ln -s /etc/nginx/sites-available/mern-vps-deployment /etc/nginx/sites-enabled/
nginx -t
systemctl reload nginx
```

Disables the default site, enables your new site via symlink, validates config, then reloads Nginx.

nginx -t must say the configuration is OK before reloading.

Step 22 - Smoke test Nginx locally

```
curl -i http://localhost/
curl http://localhost/api/notes
```

The first curl confirms Nginx is responding on port 80.

The second curl confirms Nginx can reach the API through the reverse proxy.

Step 23 - Build the React frontend on the VPS (without installing Node globally)

```
docker run --rm \
  -v "$PWD/client:/app" \
  -w /app \
  node:20-alpine \
  sh -lc "npm ci && npm run build"
```

Runs a temporary Node 20 container to build the frontend. Output should be generated in **client/dist**.

This keeps your VPS clean (no Node install) and ensures consistent Node version.

Note: If your client folder name differs, update the volume mount path.

Step 24 - Copy the built dist/ folder to Nginx web root

```
mkdir -p /var/www/mern-vps-deployment/dist
rsync -a --delete ./client/dist/ /var/www/mern-vps-deployment/dist/
chown -R www-data:www-data /var/www/mern-vps-deployment
```

Creates a stable location for static files under **/var/www**.

rsync --delete keeps the server folder identical to the latest build output.

Ownership is set to **www-data** so Nginx can read files cleanly.

Step 25 - Update Nginx config to serve frontend + proxy /api

```
nano /etc/nginx/sites-available/mern-vps-deployment

# /etc/nginx/sites-available/mern-vps-deployment
server {
  listen 80;
  server_name mernvpsdeploy.duckdns.org;

  # Serve frontend build output
  root /var/www/mern-vps-deployment/dist;
  index index.html;

  # Single Page App routing (React Router)
  location / {
    try_files $uri $uri/ /index.html;
  }

  # API reverse proxy
  location /api/ {
    proxy_pass http://127.0.0.1:5000;
    proxy_http_version 1.1;

    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
  }
}
```

Now Nginx serves your React app at **/** and still proxies the backend at **/api**.

try_files ... /index.html is required for client-side routing (so deep links work).

Step 26 - Validate and reload Nginx

```
nginx -t
systemctl reload nginx
```

Always run **nginx -t** before reloading. A syntax error can take your site down.

Step 27 - Test the domain over HTTP

```
curl -I http://mernvpsdeploy.duckdns.org
curl http://mernvpsdeploy.duckdns.org/api/notes
```

Confirms the domain resolves to your VPS and Nginx is serving content.

The API request should return the same JSON you saw via localhost earlier.

Note: If you get 502, check Docker container status and Nginx error logs.

Step 28 - Install Certbot and enable HTTPS (Let's Encrypt)

```
apt install -y certbot python3-certbot-nginx
certbot --nginx -d mernvpsdeploy.duckdns.org
```

Certbot obtains a free TLS certificate from Let's Encrypt and updates your Nginx config automatically.

During the wizard, choose the redirect-to-HTTPS option if you want all HTTP traffic to upgrade to HTTPS.

Note: Certbot will fail if DNS is not pointing correctly or if port 80 is blocked.

Step 29 - Verify HTTPS end-to-end

```
curl -I https://mernvpsdeploy.duckdns.org
curl https://mernvpsdeploy.duckdns.org/api/notes
```

Confirms TLS is active and both frontend and backend work through the public domain over HTTPS.

Note: Let's Encrypt certificates auto-renew (system timer). You can test renewal with: **certbot renew --dry-run**.

Appendix - Quick troubleshooting commands

Use these when something breaks during the steps above.

Docker / API

```
cd /opt/mern-vps-deployment
docker compose -f docker-compose.prod.yml ps
docker compose -f docker-compose.prod.yml logs -f api
```

If the container is restarting or exited, the logs will usually show the real error (missing env var, DB connection, crash).

Nginx

```
nginx -t
systemctl status nginx --no-pager
tail -n 100 /var/log/nginx/error.log
tail -n 100 /var/log/nginx/access.log
```

A common failure is **502 Bad Gateway** (Nginx cannot reach 127.0.0.1:5000). Check that the API container is Up and ports are correct.

Firewall / ports

```
ufw status
ss -lntp | head
curl -I http://localhost
```

Ports 80 and 443 must be allowed (Nginx Full) for HTTP/HTTPS. The API port should be bound to 127.0.0.1 only.